



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE CODE: ITA0404

COURSE OUTCOME COVERED

CO2 -- ASSESSMENT TOOL 1

CO2: Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic. Analyse the Correlation between the parameters in a dataset and establish the analysis (k3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

Duration: 1 Hour

Coding Challenge

Code of Conduct:

I J.Raghavi Reddy (192525158) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J.Raghavi Reddy

Reg. No: 192525158

RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30						
Accuracy of Answer	25						
Application of Knowledge	20						



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



Completeness of Response	15						
Clarity & Presentation	10						
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

1. Student Marks Analyzer

Write an R program to analyze the marks of **10 students** stored in a vector.

- Use a for() loop to process each mark.
- Classify students as: ○ **Distinction:** ≥ 75
 ○ **Pass:** 50–74 ○ **Fail:** < 50
- Use if...else control statements.
- Display the total number of students in each category.

Sample Input:

```
marks <- c(82, 45, 67, 91, 38, 76, 54, 49, 88, 72)
```

2. ATM Withdrawal Challenge

Write an R program to simulate an ATM withdrawal system.

- Start with a balance of **₹10,000**.
- Repeatedly ask the user for a withdrawal amount using a while() loop.
- Stop when the user enters 0.
- If the withdrawal amount is greater than the balance, display "**Insufficient Balance**".
- If the amount is not a multiple of ₹100, display "**Invalid Amount**". • Otherwise, deduct the amount and display the remaining balance.

Concepts: while(), if...else, break, control statements.

3. Matrix Diagonal Processing

Given a square matrix, write an R program to process its elements.



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



- Use **nested for() loops** to traverse the matrix.
- Calculate the sum of the **main diagonal**.
- Calculate the sum of the **secondary diagonal**.
- Use control statements to identify and display the diagonal elements separately. • Finally, determine which diagonal has the greater sum.

Sample Input:

```
m <- matrix(c(5, 2, 8,  
             1, 7, 4,  
             6, 3, 9), nrow = 3, byrow = TRUE)
```

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

4. Looping Over a List of Employee Records

Create an R **list** containing information about 5 employees. Each employee should have a name, department, and salary.

Write a program to:

- Use a for() loop to process each employee record.
- Display employees whose salary is greater than **₹50,000**.
- Count employees in each department.
- Find the employee with the highest salary.
- Use next to skip incomplete employee records.

Sample Structure: employees <- list(
list(name="Arun", dept="CSE", salary=55000),
list(name="Bala", dept="ECE", salary=48000),
list(name="Chitra", dept="CSE", salary=62000),
list(name="Deepa", dept="IT", salary=51000),
list(name="Ezhi", dept="ECE", salary=45000)
)

5. CREATE YOUR OWN PROBLEM FOR CODING CHALLENGE WITH SOLUTION

Coverage: for() • while() • if/else • break • next • vectors • matrices • lists • nested loops.

Top of Form

Bottom of Form

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 AT-1

NAME: J.Raghavi Reddy

FACULTY: Dr.G.Kumaresan

REG.NO: 192525158

COURSE CODE:ITA0404

COURSE NAME: Statistics with R Programming

1.

Aim: To analyze the marks of 10 students and classify them into **Distinction, Pass, and Fail** categories using for() and if...else.

Code:

```
marks <- c(82, 45, 67, 91, 38, 76, 54, 49, 88, 72)
```

```
distinction <-
```

```
0 pass <- 0 fail
```

```
<- 0
```

```
for (mark in marks) { if
```

```
(mark >= 75) { distinction
```

```
<- distinction + 1 } else if
```

```
(mark >= 50) { pass <- pass
```

```
+ 1 }
```

```
else {
```

```
fail <- fail + 1
```

```
}
```

```
}
```

```
cat("Distinction:", distinction, "\n")
```

```
cat("Pass:", pass, "\n") cat("Fail:",
```

```
fail, "\n")
```

Input:

```
marks <- c(82, 45, 67, 91, 38, 76, 54, 49, 88, 72)
```

Output:

```
Distinction: 4
```

```
Pass: 4
```

```
Fail: 2
```

Result: The marks of 10 students were successfully classified into Distinction, Pass, and Fail

categories.

2.

Aim: To simulate an ATM withdrawal system using a while() loop, if...else, and break statements.

Code:

```
balance <- 10000
```

```
while (TRUE) { amount <- as.numeric(readline("Enter withdrawal  
amount (0 to stop): "))
```

```
  if (amount == 0) {  
    cat("Transaction stopped.\n")    break  
  } else if (amount > balance) {  
    cat("Insufficient Balance\n")  }  
  else if (amount %% 100 != 0) {  
    cat("Invalid Amount\n")    } else {  
    balance <- balance - amount  
    cat("Withdrawal successful\n")  
    cat("Remaining Balance:", balance, "\n")  
  }  
}
```

Input:

350

3000

0

Output:

Enter withdrawal amount: 2500 Enter withdrawal
amount: 350

Enter withdrawal amount: 3000

Enter withdrawal amount: 0

2500

Result:

The ATM withdrawal system was successfully implemented using while(), if...else, and break.

3.

Aim: To calculate the sums of the main and secondary diagonals of a square matrix using nested for() loops and control statements.

Code:

```
m <- matrix(c(5, 2, 8,
              1, 7, 4,
              6, 3, 9),
            nrow = 3, byrow = TRUE)

n <- nrow(m)

main_sum <- 0 secondary_sum
<- 0

cat("Main Diagonal Elements: ")

for (i in 1:n) { for (j in 1:n) { if (i
== j) { cat(m[i, j], " ")
main_sum <- main_sum + m[i, j]
}
}
}

cat("\nSecondary Diagonal Elements: ")

for (i in 1:n) { for (j
in 1:n) { if (i + j == n + 1) { cat(m[i, j], "
")
secondary_sum <- secondary_sum +
m[i, j]
}
}
}

cat("\nMain Diagonal Sum:", main_sum, "\n") cat("Secondary
Diagonal Sum:", secondary_sum, "\n")

if (main_sum > secondary_sum) { cat("Main
diagonal has the greater sum.\n") } else if
(secondary_sum > main_sum) { cat("Secondary
diagonal has the greater sum.\n") } else {
cat("Both diagonals have equal sums.\n")
}
```

Output:

Main Diagonal Elements: 5 7 9

Secondary Diagonal Elements: 8 7 6

Main Diagonal Sum: 21 Secondary

Diagonal Sum: 21

Both diagonals have equal sums. **Result:**

The main and secondary diagonal elements and their sums were successfully calculated using nested loops.

4.

Aim: To process employee records using a for() loop, identify employees earning more than ₹50,000, count employees by department, find the highest-paid employee, and skip incomplete records using next.

Code:

```
employees <- list( list(name="Arun",
dept="CSE", salary=55000), list(name="Bala",
dept="ECE", salary=48000), list(name="Chitra",
dept="CSE", salary=62000), list(name="Deepa",
dept="IT", salary=51000), list(name="Ezhil",
dept="ECE", salary=45000)
)

dept_count <- list() highest_salary
<- 0 highest_employee <- ""

cat("Employees with salary greater than Rs.50000:\n")

for (emp in employees) {

  if (is.null(emp$name) || is.null(emp$dept) || is.null(emp$salary)) {
next
  }

  if (emp$salary > 50000) { cat(emp$name, "-",
emp$dept, "-", emp$salary, "\n")
  }

  if (is.null(dept_count[[emp$dept]])) {
dept_count[[emp$dept]] <- 1
  } else { dept_count[[emp$dept]] <- dept_count[[emp$dept]]
+ 1
  }

  if (emp$salary > highest_salary) {
```

```

highest_salary <- emp$salary   highest_employee
<- emp$name
}
}

cat("\nEmployees in each department:\n")

for (dept in names(dept_count)) {
cat(dept, ":", dept_count[[dept]], "\n")
}

cat("\nHighest Salary Employee:\n") cat(highest_employee,
"- ", highest_salary, "\n")

```

Output:

Employees with salary greater than Rs.50000:

Arun - CSE - 55000

Chitra - CSE - 62000

Deepa - IT - 51000

Employees in each department:

CSE : 2

ECE : 2

IT : 1

Highest Salary Employee:

Chitra - 62000

Result:

The employee records were successfully processed using lists, loops, if...else, and next.

5.

Problem Statement:

Store electricity units in a vector.

- Use a for() loop to process each household.
- Calculate the bill using the following rates:
 - 0–100 units: ₹2 per unit ○
 - 101–200 units: ₹3 per unit ○
 - Above 200 units: ₹5 per unit
- Use if...else to determine the bill.
- Use while() to display the bills.
- Use break when all households are processed.
- Use next to skip a household with 0 units.
- Store the results in a matrix.

- Use a list to store summary information.

Aim: To calculate and analyze electricity bills using vectors, matrices, lists, for(), while(), if...else, break, next, and nested loops.

Code:

```
units <- c(80, 150, 0, 220, 180)
```

```
bills <- numeric(5)
```

```
# Calculate bills using for loop for
(i in 1:length(units)) {
```

```
  if (units[i] == 0) {
next  }
```

```
  if (units[i] <= 100) {
bills[i] <- units[i] * 2  } else
if (units[i] <= 200) {
bills[i] <- units[i] * 3  } else
{  bills[i] <- units[i] *
5
  }
}
```

```
# Create matrix result <- matrix(c(units, bills), nrow
= 5, ncol = 2)
```

```
colnames(result) <- c("Units", "Bill")
```

```
cat("Electricity Bill Details:\n") print(result)
```

```
# Nested loops to display matrix cat("\nMatrix
Elements:\n")
```

```
for (i in 1:nrow(result)) {
for (j in 1:ncol(result)) {  cat(result[i,
j], " ")
}  cat("\n")
}
```

```
# while loop i
<- 1
```

```

cat("\nBills using while loop:\n")

while (TRUE) {

  if (i > length(bills)) {    break
  }

  if (bills[i] == 0) {    i
<- i + 1    next
  }

  cat("House", i, "Bill: Rs.", bills[i], "\n")    i
<- i + 1 }

# Create list summary <- list( Total_Houses
= length(units),
  Total_Units = sum(units), Total_Bill
= sum(bills),
  Highest_Bill = max(bills)
)

cat("\nSummary:\n") print(summary)

```

Output:

Electricity Bill Details:

	Units	Bill
[1,]	80	160
[2,]	150	450
[3,]	0	0
[4,]	220	1100
[5,]	180	540

Matrix Elements:

```

80 160
150 450
0 0
220 1100
180 540

```

Bills using while loop:

```

House 1 Bill: Rs. 160
House 2 Bill: Rs. 450
House 4 Bill: Rs. 1100
House 5 Bill: Rs. 540

```

Summary:

\$Total_Houses

[1] 5

\$Total_Units

[1] 630

\$Total_Bill

[1] 2250

\$Highest_Bill

[1] 1100

Result:

The electricity bill analyzer was successfully implemented using vectors, matrices, lists, for(), while(), nested loops, if...else, break, and next.